FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Ingeniería de Sistemas - UNICEN

Base de Datos II

SQL Avanzado

Funciones de Ventana

Ing. Gustavo A. Correa • 2026

Que vamos a ver ?



1. Expresiones de Tabla (CTE)

Qué es una Common Table Expression en SQL y cómo estructurar consultas temporales.



2. Recursividad en SQL

Uso de recursión para recorrer estructuras jerárquicas y procesar datos recursivos.



3. Agrupamientos Múltiples

Operaciones avanzadas utilizando las cláusulas CUBE, ROLL UP y Grouping Sets.



4. Funciones de Ventana

Cálculos avanzados sobre conjuntos de filas sin agrupar o colapsar registros.

¿Qué es una CTE?

SQL · CONSULTAS AVANZADAS

Common Table Expression

Expresión de tabla común

Es un conjunto de resultados temporal, con nombre propio, que funciona como una "tabla virtual" definida al comienzo de una consulta mediante la cláusula `WITH`.

Temporal

Desaparece automáticamente al terminar la consulta.

Con nombre

Se referencia fácilmente como una tabla tradicional.

No se guarda

No crea objetos persistentes en la base de datos.

```sql
WITH clientes_activos AS
(
  SELECT id, nombre
  FROM clientes
  WHERE estado = 'activo'
)

SELECT *
FROM clientes_activos;
```

Sintaxis mínima:

```sql
WITH nombre AS ( consulta )
```

Anatomía de una CTE

```sql
WITH ventas_2025 AS (
  SELECT vendedor_id,
  SUM(importe) AS total
  FROM ventas
  WHERE anio = 2025
  GROUP BY vendedor_id
)

SELECT v.nombre, m.total
FROM vendedores v
JOIN ventas_2025 m
 ON m.vendedor_id = v.id;
```

1

WITH nombre AS (...)

Declara la CTE y le da un nombre, siempre al inicio de la consulta.

2

Consulta interna

Define el resultado temporal: aquí, el total vendido por vendedor durante 2025.

3

Consulta principal

Usa la CTE como una tabla más: se puede unir con JOIN, filtrar u ordenar.

¿Por qué usarlas?

Legibilidad

Dividen consultas complejas en pasos con nombre, que se leen de arriba hacia abajo.

Reutilización

Una misma CTE puede referenciarse varias veces sin repetir la subconsulta.

Encadenamiento

Varias CTEs separadas por comas, cada una construida sobre la anterior.

Recursión

Con WITH RECURSIVE se recorren jerarquías: organigramas, categorías, rutas.

Soportadas por PostgreSQL, MySQL 8+, SQL Server, Oracle y SQLite.

Consultas Recursivas

Resolviendo Jerarquías en SQL

Concepto Base

Una **CTE recursiva** es una consulta que se referencia a sí misma repetidamente para recorrer y procesar datos jerárquicos.

- Ideal para organigramas y dependencias
- Estructuras de categorías anidadas
- Sistemas de navegación y rutas de red

La Analogía

"Funciona como las muñecas rusas (Matryoshkas). Abres una para encontrar otra más pequeña, hasta llegar a la última pieza."



Estructura Clave

Ancla, Unión y Recursión en CTEs Recursivas

Componentes

1. Miembro Ancla (Base)

El punto de partida o consulta base inicial que siembra el primer conjunto de resultados.

2. UNION ALL

El operador que combina los resultados acumulados de la base con las iteraciones recursivas.

3. Miembro Recursivo

El ciclo de consulta que se ejecuta repetidamente referenciando el resultado anterior de la misma CTE.

Código de Ejemplo

```
WITH RECURSIVE cte_name AS (
  -- 1. Miembro Ancla (Base)
  SELECT id, padre_id, nombre
  FROM componentes WHERE padre_id IS NULL

  UNION ALL

  -- 2. Miembro Recursivo
  SELECT c.id, c.padre_id, c.nombre
  FROM componentes c
  INNER JOIN cte_name r ON c.padre_id =
  r.id
)
```

Consultas Recursivas

Ejemplo práctico: Generar números secuenciales del 1 al 10

Código SQL

```sql
WITH RECURSIVE ejemplo AS (
  -- Semilla (Caso Base)
  SELECT 1 n
  UNION ALL
  -- Término Recursivo
  SELECT n+1 FROM ejemplo
)
SELECT n FROM ejemplo LIMIT 10;
```

Funcionamiento

1. Inicialización (Semilla):
Se ejecuta la consulta base `SELECT 1` que genera el primer registro y define el tipo de dato para la columna `n`.

2. Unión Recursiva:
La segunda consulta toma los resultados previos e incrementa su valor (`n+1`) de manera sucesiva y acumulativa.

3. Límite de Parada:
La instrucción `LIMIT 10` en la consulta final restringe la recursión deteniendo el proceso al alcanzar 10 filas.

Consultas Recursivas

Ejemplo: listar el coordinador principal (jefe) y sus voluntarios coordinados

Código SQL

```sql
WITH RECURSIVE voluntario_coord AS (
  -- Semilla (Caso Base)
  SELECT 1 n, id_coordinador, nro_voluntario,
  nombre, apellido
  FROM voluntario WHERE id_coordinador IS NULL

  UNION ALL

  -- Término Recursivo
  SELECT n+1, v.id_coordinador,
  v.nro_voluntario, v.nombre, v.apellido
  FROM voluntario_coord vc, voluntario v
  WHERE vc.nro_voluntario = v.id_coordinador
)

SELECT * FROM voluntario_coord WHERE n < 3;
```

Explicación

1. Semilla (Caso Base):
Obtiene al jefe máximo que no tiene coordinador (`IS NULL`) asignándole el nivel inicial `n = 1`.

2. Recursión:
Une sucesivamente a cada voluntario con su respectivo coordinador, sumando `n + 1` en cada paso.

3. Límite de Niveles:
La condición final `WHERE n < 3` restringe el árbol jerárquico para mostrar únicamente los niveles 1 y 2.

Consultas Recursivas

Documentación oficial y recursos de aprendizaje avanzados

Para ir por más:

Explorá la documentación oficial de PostgreSQL para comprender en profundidad las reglas de resolución de nombres, ciclos de recursión y optimizaciones de rendimiento al utilizar expresiones de tabla comunes (CTE).

Enlace de Referencia

`https://www.postgresql.org/docs/current/queries-with.html`

Consultas Recursivas

Espacio abierto para dudas, comentarios y debate



¿Preguntas?

Repasemos los conceptos clave de la sesión:

- Estructura del Caso Base (Semilla) y Término Recursivo
- Condición de parada y prevención de bucles infinitos
- Casos de uso reales y optimización con CTEs en PostgreSQL

GROUPING SETS, CUBE, y ROLLUP

GROUPING en consultas avanzadas

Objetivo Principal

Permite realizar operaciones de agrupación más complejas utilizando el concepto de **conjuntos de agrupación**.

Facilita la creación de informes multidimensionales en una única consulta estructurada, evitando múltiples UNION ALL.

Funcionamiento

Agrupa por separado por cada conjunto de agrupación especificado en la consulta.

Las funciones de agregación se calculan de manera independiente para cada grupo.

`Equivalente a aplicar GROUP BY de forma individual.`

GROUPING SETS, CUBE, y ROLLUP

El Desafío

¿Cómo calcular la suma de horas aportadas por tarea e institución y el total general ?

(id_tarea, id_institución, sum, count)

¡En una única consulta!

Evita múltiples UNION ALL combinando diferentes niveles de agrupación de forma eficiente.

Consulta con GROUPING SETS

```
SELECT id_tarea,
    id_institucion,
    sum(horas_aportadas),
    count(*)
FROM voluntario
WHERE id_institucion IS NOT NULL
GROUP BY GROUPING SETS (
  id_tarea, id_institucion, ()
)
ORDER BY id_tarea, id_institucion;
```

GROUPING SETS, CUBE, y ROLLUP

Operaciones de agregación: El modificador **ROLLUP**

Sintaxis de ROLLUP

```
SELECT
    c1,
    c2,
    c3,
    [aggr_func](c4)
FROM table
GROUP BY ROLLUP (
    c1, c2, c3
);
```

Grupos Generados

ROLLUP

Realiza una agregación jerárquica reduciendo dimensiones:

→ `(c1, c2, c3)`
Máximo nivel de detalle especificado.

→ `(c1, c2)`
Agrupación intermedia eliminando c3.

→ `(c1)`
Agrupación sobre el primer elemento.

→ `()`
Gran Total general (sin agrupar).

GROUPING SETS, CUBE, y ROLLUP

Ejemplo práctico: El modificador **ROLLUP**

El Desafío

¿Cómo calcular la suma de horas aportadas por tarea e institución y el total general?

¡En una única consulta!

El modificador ROLLUP permite agrupar jerárquicamente de derecha a izquierda, obteniendo los subtotales y el gran total de forma sumamente eficiente.

Consulta con ROLLUP

```
SELECT
    id_institucion,
    id_tarea,
    sum(horas_aportadas)
FROM voluntario
WHERE id_institucion IS NOT NULL
GROUP BY ROLLUP (
    id_tarea, id_institucion
)
ORDER BY id_institucion, id_tarea;
```

Diferencias de Agregación

Comparativa de GROUPING SETS, ROLLUP y CUBE

GROUPING SETS

Te deja **decir explícitamente** qué agrupaciones quieres. Es la forma más flexible.

Ejemplo de sintaxis

```
GROUP BY GROUPING SETS
((region, producto),
region, ())
```

ROLLUP (a, b, c)

Es un **atajo jerárquico** que realiza una agregación de arriba hacia abajo reduciendo dimensiones.

Combinaciones

- → `(a, b, c)`
- → `(a, b)`
- → `(a)`
- → `()`

CUBE (a, b, c)

Es el método **exhaustivo** que genera todas las agrupaciones posibles entre las columnas.

Combinaciones

`(a,b,c), (a,b), (a,c), (b,c), (a), (b), (c), ()`

Equivalencias

Estructuras equivalentes expresadas mediante GROUPING SETS

Ejemplo con dos columnas

ROLLUP (a, b)

≡ GROUP BY GROUPING SETS (
  (a, b), (a), ()
)

CUBE (a, b)

≡ GROUP BY GROUPING SETS (
  (a, b), (a), (b), ()
)

Ejemplo con tres columnas

ROLLUP (a, b, c)

≡ GROUP BY GROUPING SETS (
  (a, b, c), (a, b), (a), ()
)

CUBE (a, b, c)

≡ GROUP BY GROUPING SETS (
  (a,b,c), (a,b), (a,c), (b,c),
  (a), (b), (c), ()
)

¿Preguntas o Dudas?

Repaso de conceptos de agregaciones complejas

GROUPING SETS

?¿Quedó claro cómo definir explícitamente cada nivel de agregación?
?¿Cuándo conviene usarlo en lugar de múltiples consultas con `UNION ALL`?

ROLLUP

?¿Se entiende la estructura jerárquica de reducción de derecha a izquierda?
?¿Comprenden cómo calcula los subtotales y el gran total general en una sola pasada?

CUBE

?¿Tienen claro por qué genera un crecimiento exponencial de agrupaciones (2^n)?
?¿Cuándo se justifica el costo de procesamiento de un análisis multidimensional exhaustivo?

Funciones de Ventana

Origen y Estándar

Son herramientas estandarizadas en **SQL:2003**.

La versión **8.4 de PostgreSQL** ya contaba con soporte para estas funciones, convirtiéndose en el primer DBMS de código abierto en lograrlo.

Lamentablemente muy poco utilizadas por falta de conocimiento.

Sintaxis Distintiva

Se distinguen claramente de las funciones de agregación tradicionales basadas en **GROUP BY**.

Permiten realizar cálculos avanzados sin colapsar o agrupar las filas individuales de la consulta, manteniendo la identidad de cada registro original.

GROUP BY - Aclaración

Consulta Permitida

¿Por qué en algunos casos puedo seleccionar más campos que los que están en el ***group by***?

```
SELECT i.id_institucion,
  nombre_institucion,
  count(*)
FROM institucion i
  JOIN voluntario v
    ON (i.id_institucion
 = v.id_institucion)
GROUP BY i.id_institucion
```

Consulta con Error

¿Por qué este caso en particular produce un error de sintaxis y agrupación en el DBMS?

```
SELECT v.id_institucion,
  nombre_institucion,
  count(*)
FROM institucion i
  JOIN voluntario v
    ON (i.id_institucion
 = v.id_institucion)
GROUP BY v.id_institucion
```

Mucho cuidado: la consulta de la izquierda sólo funciona en PostgreSQL

Cómo hacemos si queremos...

...listar para cada voluntario las horas que aporta, la tarea que realiza y el **promedio de hs** de dicha tarea?

id_tarea	nro_voluntario	horas_aportadas	promedio
AD_VP	102	17000	**17000**
AD_VP	101	17000	**17000**
FI_ACCOUNT	113	6900	**7920**
FI_ACCOUNT	112	7800	**7920**
FI_ACCOUNT	111	7700	**7920**
FI_ACCOUNT	110	8200	**7920**
FI_ACCOUNT	109	9000	**7920**
FI_MGR	108	12000	**12000**
HR_REP	203	6500	**6500**

Funciones de Ventana



Cláusula OVER

Siempre contiene una cláusula **OVER** después del nombre y los argumentos de la función de ventana.



Definición de Filas

OVER indica cómo se dividen las filas de la consulta para que las procese la función de ventana.



PARTITION BY

especifica dividir las filas en grupos que comparten los mismos valores.

Funciones de Ventana

El Problema

¿Cómo comparar las horas aportadas de cada voluntario con las horas aportadas en promedio para cada tarea?

Para resolver esto de forma tradicional se requeriría una subconsulta compleja. Con una función de ventana, podemos mantener el detalle de cada fila original y anexar el cálculo grupal de forma directa.

Consulta SQL

```
SELECT id_tarea, nro_voluntario,
       horas_aportadas,
       avg(horas_aportadas) OVER
       (PARTITION BY id_tarea)
FROM voluntario;
```

La instrucción **OVER (PARTITION BY ...)** divide los datos en grupos lógicos según la tarea, permitiendo al promedio calcularse por grupo sin alterar la granularidad de los voluntarios.

Funciones de Ventana

El Desafío

¿Dentro de cada tarea cómo es el ranking de horas aportadas de cada voluntario?

Para resolver esto, la función de ventana rank() nos permite generar una clasificación ordinal dentro de cada grupo de tareas, ordenando a los voluntarios según su esfuerzo de mayor a menor.

Consulta SQL

```
SELECT id_tarea, nro_voluntario,
    horas_aportadas,
    rank() OVER (
        PARTITION BY id_tarea
        ORDER BY horas_aportadas
        DESC)

FROM voluntario;
```

La instrucción **PARTITION BY** agrupa los registros por tarea, mientras que el **ORDER BY ... DESC** define la jerarquía del ranking dentro de cada grupo.

Funciones de Ventana

El Desafío

¿Cómo comparar las horas aportadas de cada voluntario con el promedio acumulado dentro de cada tarea?

Para resolver esto, la combinación de la función agregada **avg()** con **ORDER BY** calcula un promedio acumulativo paso a paso dentro de cada partición.

Consulta SQL

```
SELECT id_tarea, nro_voluntario,
   horas_aportadas,
   avg(horas_aportadas) OVER (
      PARTITION BY id_tarea
      ORDER BY horas_aportadas)

FROM voluntario;
```

La cláusula **ORDER BY** dentro del **OVER** establece un marco de ventana acumulativo desde la primera fila hasta la actual de la partición.

Funciones de Ventana

Estructura de la Sintaxis

```
window_function ([arg1, arg2,..]) OVER
   [PARTITION BY partition_expression]
   [ORDER BY sort_expression
      [ASC | DESC] [NULLS {FIRST | LAST}]]
```

PARTITION BY agrupa las filas en particiones para un cálculo aislado.
ORDER BY ordena las filas de cada partición.

Algunas funciones de Ventana más comunes

LAG (expr [,o,d])	Valor de la fila anterior.	`LAG(monto) OVER (...)`
LEAD (expr [,o,d])	Valor de la fila siguiente.	`LEAD(monto) OVER (...)`
ROW_NUMBER ()	Índice secuencial por partición (sin iguales).	`ROW_NUMBER() OVER (...)`
RANK ()	Ranking con huecos cuando hay iguales (1,1,3).	`RANK() OVER (ORDER BY monto DESC)`
DENSE_RANK ()	Ranking sin huecos (1,1,2).	`DENSE_RANK() OVER (...)`

Pero se puede complicar mucho más

Ejemplos prácticos utilizando **SUM()** y **AVG()** combinados con la cláusula **OVER** para acumulados y medias móviles.

Cálculo Acumulado

También conocido como "running total". Suma los valores de forma progresiva por cada partición de datos.

```
SUM(monto) OVER (
  PARTITION BY region
  ORDER BY fecha
  ROWS BETWEEN
    UNBOUNDED PRECEDING
    AND CURRENT ROW
)
```

Establece que el marco de cálculo inicia desde la primera fila de la partición hasta la fila actual.

Media Móvil (3)

Calcula el promedio utilizando un marco de ventana deslizante limitado para suavizar fluctuaciones.

```
AVG(monto) OVER (
  ORDER BY fecha
  ROWS BETWEEN
    2 PRECEDING
    AND CURRENT ROW
)
```

El marco se restringe exclusivamente a las 2 filas anteriores y la fila actual, promediando 3 valores en total.

Resumen y Preguntas

Conceptos Clave

Un repaso rápido de lo aprendido sobre Funciones de Ventana:

Cláusula OVER

Define el marco o partición de los datos antes de aplicar la función.

LAG / LEAD

Permiten acceder de forma sencilla a valores de filas anteriores o posteriores.

Rankings

ROW_NUMBER, RANK y DENSE_RANK para ordenar y secuenciar.

Acumulados

Uso de ROWS BETWEEN para limitar el rango de cálculo.

Consultas Frecuentes

Preguntas recurrentes de los alumnos durante las clases:

¿RANK vs DENSE_RANK?

RANK deja huecos en empates (1,1,3); DENSE_RANK es correlativo y continuo (1,1,2).

¿Y el rendimiento?

Motor pensado para particionar y trabajar con conjuntos.